

C++ Queue

Queue in C++ is a type of data structure that is designed to work as a First In First Out (FIFO) data container.

With Queue the Data entered from one side of a queue is extracted from the other side of a queue in a FIFO manner.

C++ Queue Declaration

```
queue < datatype > queue_Name
```

For example,

```
queue<int> queue1
```

List of functions associated with Queue

1. empty()

This function is used to check whether a queue is empty or filled. The return value of the function = true if the queue is empty with no element inside it. The return value of the function = false if the queue is non-empty.

2. size()

This function is used to check the size or number of member elements of the queue.

3. front()

This function can be used to get details or references related to the front element of the queue. As the queue is First In First Out(FIFO) container, the front element is the oldest element present in the queue container.

4. back()

This function can be used to get details or references related to the last element of the queue. As the queue is the First In First Out(FIFO) container, the last element is nothing but the newest element present in the queue container.

5. push(value)

This function can be used to insert a new element in the queue. This function inserts a new element after its last inserted element. The process to insert a new element in the queue is called enqueue operation.

6. pop()

This function is used to remove an element from the queue container. The element removed by this function is the oldest element. After using this function, the size of the queue will be reduced by one. The process is also called a dequeue operation.

Example: Queue in C++

```
// including required header files
#include <iostream>
#include <queue> // header file to use queue functionalities in C++
using namespace std;
int main()
{
    // declaration of queue named queue1
    queue<int> queue1;
    // inserting element in the queue container
    queue1.push(1);
    queue1.push(2);
    queue1.push(3);
    queue1.push(4);
    queue1.push(5);
    // Removing/Extracting the content from the queue container
    while (!queue1.empty()) {
        cout << ' ' << queue1.front();
        queue1.pop();
    }
    return 0;
}
```

1 2 3 4 5